

Theory of Programming Languages

2. Lambda Calculus

Kihong Heo



Lambda Calculus 람다계산법

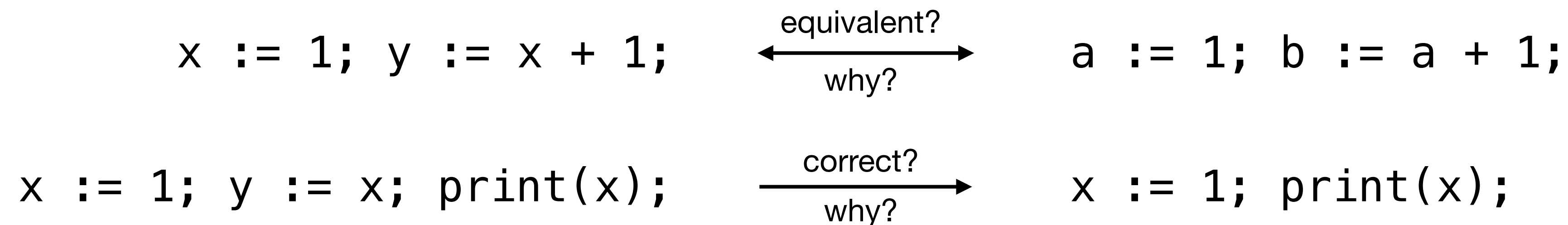
- Introduced by Alonzo Church (1935)
- Answer to the grand question: “What does computable function mean?”
 - Recursive function (Gödel), Turing machine (Turing), Lambda calculus (Church)
 - All three definitions are equivalent
 - “Any computable functions are simulated by a Turing machine” (Church-Turing Thesis)
- Foundation of programming languages
- Syntax (문법구조): $E \rightarrow x \mid \lambda x.E \mid E E$

Scope 유효범위

- x is said to be **bound** when it occurs in the body E of an abstraction $\lambda x . E$
- x is said to be **free** if it is not bound
- Example
 - $\lambda x . x y$
 - $\lambda x . \lambda y . y (\lambda x . x) x$

Semantics의미구조

- Semantics: concern with the **meaning** of grammatically correct programs
 - interpreters execute programs w.r.t. the **semantics**
 - compilers translate programs w.r.t. the **semantics**
 - analyzers analyze programs w.r.t. the **semantics**



Different Styles of Semantics

- **Operational Semantics** (과정형/어떻게 의미구조): “**How** to compute the execution result”
 - so-called transitional style (상태전이식)
 - $3 * (2 + 1) : 3 * (2 + 1) \rightarrow 3 * 3 \rightarrow 9$
- **Denotational Semantics** (직접형/무엇 의미구조): “**What** the execution result is”
 - so-called compositional style (조립식)
 - $3 * (2 + 1) : 9$
- ...

Different approaches for different purposes and languages

Operational Semantics 과정형 의미구조

- Program semantics is a series of machine state transitions
 - Big-step (큰걸음): how the **overall results** of executions are obtained
 - Small-step (잔걸음): how the **individual steps** of the computations take place
- In this lecture, we will mainly use small-step operational semantics

Big-step

$$\frac{\frac{\frac{2 \times 2 \Rightarrow 4}{2 \times 2 \times 2 \Rightarrow 8} \quad \frac{4 \times 2 \Rightarrow 8}{2 + 1 \Rightarrow 3} \quad \frac{8 \times 3 \Rightarrow 24}}{(2 \times 2 \times 2) \times (2 + 1) \Rightarrow 24}}{\text{why?}}$$

Small-step

$$\begin{aligned} & (2 \times 2 \times 2) \times (2 + 1) \\ & \rightarrow (4 \times 2) \times (2 + 1) \\ & \rightarrow 8 \times (2 + 1) \\ & \rightarrow 8 \times 3 \\ & \rightarrow 24 \end{aligned} \quad \text{so?}$$

Small-Step Semantics 작은걸음 의미구조

- Redex (reducible expression): $(\lambda x.E_1) E_2$
- β -reduction: $(\lambda x.E_1) E_2 \rightarrow E_1\{E_2/x\}$
- Different reduction strategies: full β -reduction, normal order, call-by-value (CBV), call-by-name (CBN), etc
- A term is in normal form if it cannot be reduced by β -reduction

$$(\lambda x.x)(\lambda y.y\ y)(\lambda z.z) \rightarrow (\lambda y.y\ y)(\lambda z.z) \rightarrow (\lambda z.z)(\lambda z.z) \rightarrow (\lambda z.z)$$

Example: Full β -Reduction

- Any redex can be reduced at any time

$$\frac{}{(\lambda x.E_1) E_2 \rightarrow E_1\{E_2/x\}} \quad \frac{E_1 \rightarrow E'_1}{E_1 E_2 \rightarrow E'_1 E_2} \quad \frac{E_2 \rightarrow E'_2}{E_1 E_2 \rightarrow E_1 E'_2} \quad \frac{E \rightarrow E'}{\lambda x.E \rightarrow \lambda x.E'}$$

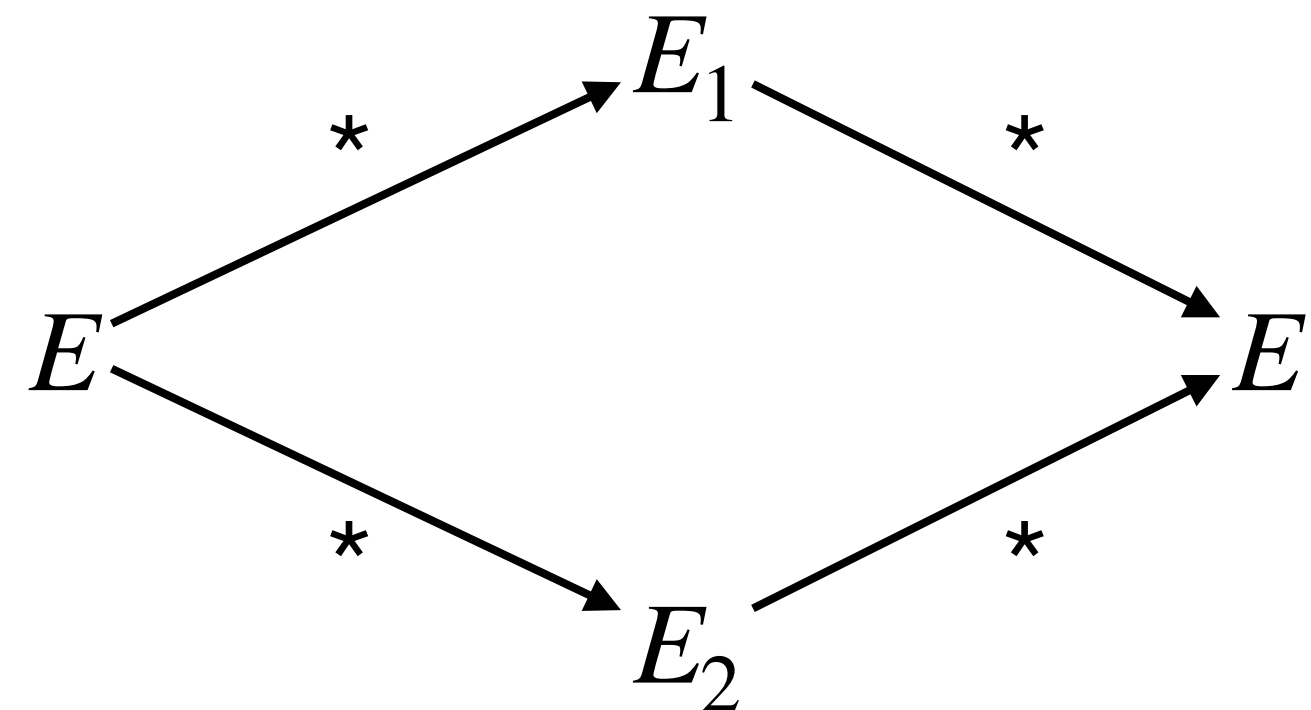
- Example

$$(\lambda x.x)((\lambda y.y)(\lambda z.(\lambda w.w)z))$$

Confluence_{만남}

Theorem (Church-Rosser). Let E be an expression. For some expressions E_1 and E_2 , if $E \rightarrow^* E_1$ and $E \rightarrow^* E_2$ then

$$\exists E' . E_1 \rightarrow^* E' \wedge E_2 \rightarrow^* E'$$



Example: Call-by-name (CBN)

- Reduce the leftmost outermost redex
 - Function application even when its argument is not a value (lazy evaluation)
 - Used in languages such as Haskell, OCaml (with “lazy”), etc

$$\frac{}{(\lambda x.E_1) E_2 \rightarrow E_1\{E_2/x\}} \quad \frac{E_1 \rightarrow E'_1}{E_1 E_2 \rightarrow E'_1 E_2}$$

- Example

$(\lambda x.x)((\lambda y.y)(\lambda z.(\lambda w.w)z))$

$(\lambda x.\lambda y.x)((\lambda z.z)(\lambda w.w))$

$((\lambda x.x)(\lambda y.y))((\lambda z.z)((\lambda w.w)(\lambda v.v)))$

Example: Call-by-value (CBV)

- Reduce the leftmost outermost redex when its right-hand side is a value ($\lambda x.E$)
- Arguments evaluated fully before function applications (eager evaluation)
- Widely used in mainstream languages such as OCaml, C/C++, Java, etc

$$\frac{}{(\lambda x.E) v \rightarrow E\{v/x\}} \quad \frac{E_1 \rightarrow E'_1}{E_1 E_2 \rightarrow E'_1 E_2} \quad \frac{E_2 \rightarrow E'_2}{v_1 E_2 \rightarrow v_1 E'_2}$$

- Example

$(\lambda x.x)((\lambda y.y)(\lambda z.(\lambda w.w)z))$

$(\lambda x.\lambda y.x)((\lambda z.z)(\lambda w.w))$

Cf. Big-Step Semantics 큰걸음 의미구조

- $E \Downarrow v$: Term E evaluates to value v
- Rule (CBV):

$$\frac{}{\lambda x.E \Downarrow \lambda x.E} \quad \frac{E_1 \Downarrow \lambda x.E_3 \quad E_2 \Downarrow v \quad E_3\{v/x\} \Downarrow v'}{E_1 E_2 \Downarrow v'}$$

Theorem (Equivalence). For every term E and value v ,

$$E \Downarrow v \iff E \rightarrow^* v$$

Substitution

- The main part of the β -reduction: $E_1\{E_2/x\}$
- Example: $(\lambda x.x\ x)(\lambda y.y) \rightarrow (x\ x)\{\lambda y.y/x\} = (\lambda y.y)(\lambda y.y)$
- Looks simple but actually quite tricky

Substitution: First Attempt

- Naive definition

$$y\{E/x\} = \begin{cases} E & \text{if } y = x \\ y & \text{otherwise} \end{cases}$$

$$(\lambda y.E_1)\{E/x\} = \lambda y.E_1\{E/x\}$$

$$(E_1 E_2)\{E/x\} = (E_1\{E/x\}) (E_2\{E/x\})$$

- Problem?
- Example: $(\lambda x.x)\{y/x\}$
- Why? Incorrectly change bound variables
- How to fix? Don't substitute bound variables

Substitution: Second Attempt

- Idea: stop substitution for bound variables

$$y\{E/x\} = \begin{cases} E & \text{if } y = x \\ y & \text{otherwise} \end{cases}$$

$$(\lambda y.E_1)\{E/x\} = \begin{cases} \lambda y.E_1 & \text{if } y = x \\ \lambda y.E_1\{E/x\} & \text{otherwise} \end{cases}$$

$$(E_1 E_2)\{E/x\} = (E_1\{E/x\}) (E_2\{E/x\})$$

- Problem?
- Example: $(\lambda z.x)\{z/x\}$
- Why? Free variables can become bound variables (so-called variable capture)
- How to fix? Don't substitute if the param = a free variable of the target expression

Free Variable

- Variables not bound in an abstraction

$$FV(x) = \{x\}$$

$$FV(E_1 \ E_2) = FV(E_1) \cup FV(E_2)$$

$$FV(\lambda x.E) = FV(E) \setminus \{x\}$$

Substitution: Third Attempt

- Avoid variable capture

$$y\{E/x\} = \begin{cases} E & \text{if } y = x \\ y & \text{otherwise} \end{cases}$$

$$(\lambda y.E_1)\{E/x\} = \begin{cases} \lambda y.E_1 & \text{if } y = x \\ \lambda y.E_1\{E/x\} & \text{if } y \neq x \wedge y \notin FV(E) \end{cases}$$

$$(E_1 E_2)\{E/x\} = (E_1\{E/x\}) (E_2\{E/x\})$$

- Problem?
- Example: $(\lambda y.x y)\{(y z)/x\}$
- Why? no applicable rule
- But, the name of the bound variable y really matter?

α -Conversion

- Terms that differ only in the bound variable names are interchangeable in all contexts
- Two terms are equivalent up to renaming of bound variables

$$\lambda x.x \ y = \lambda z.z \ y$$

Substitution: Final Version

- Introduce a fresh variable z when necessary

$$y\{E/x\} = \begin{cases} E & \text{if } y = x \\ y & \text{otherwise} \end{cases}$$

$$(\lambda y.E_1)\{E/x\} = \begin{cases} \lambda y.E_1 & \text{if } y = x \\ \lambda y.E_1\{E/x\} & \text{if } y \neq x \wedge y \notin FV(E) \\ (\lambda z.E_1\{z/y\})\{E/x\} & \text{if } y \neq x \wedge y \in FV(E) \end{cases}$$

$$(E_1 E_2)\{E/x\} = (E_1\{E/x\}) (E_2\{E/x\})$$

- Conventionally, implicitly assume α -conversion

$$y\{E/x\} = \begin{cases} E & \text{if } y = x \\ y & \text{otherwise} \end{cases}$$

$$(\lambda y.E_1)\{E/x\} = \lambda y.E_1\{E/x\} \quad \text{if } y \neq x \wedge y \notin FV(E)$$

$$(E_1 E_2)\{E/x\} = (E_1\{E/x\}) (E_2\{E/x\})$$

Evaluation Context Semantics 실행문맥 의미구조

- A convenient way to define operational semantics
 - Separate rules into computation (계산) and congruence (나란한) rules
- Operational semantic rule = program rewriting rule
- Where to rewrite? By the evaluation contexts
- How to rewrite? By the rewriting rules
- **Example** $((\lambda x.x)(\lambda y.y))((\lambda z.z)(\lambda w.w)) \rightarrow (\lambda y.y)((\lambda z.z)(\lambda w.w)) \rightarrow (\lambda y.y)(\lambda w.w) \rightarrow \lambda w.w$

$$\frac{}{(\lambda x.E) v \rightarrow E\{v/x\}}$$

$$\frac{E_1 \rightarrow E'_1}{E_1 E_2 \rightarrow E'_1 E_2}$$

$$\frac{E_2 \rightarrow E'_2}{v_1 E_2 \rightarrow v_1 E'_2}$$

Evaluation Context 실행문맥

- Evaluation context K : program with rewrite location
- Term with $[]$ (hole), which will be rewritten with a new expression
- K is defined using a grammar
- $K[]$ (or simply K) : program with rewriting target
- $K[E]$: program where the hole has been filled with E
- **Example:** $((\lambda x.x)(\lambda y.y))((\lambda z.z)(\lambda w.w)) \rightarrow (\lambda y.y)((\lambda z.z)(\lambda w.w)) \rightarrow (\lambda y.y)(\lambda w.w) \rightarrow \lambda w.w$

Evaluation Context for Lambda Calculus

- Evaluation contexts for CBV

$$\begin{array}{lcl} E & \rightarrow & x \mid \lambda x.E \mid E E \\ v & \rightarrow & \lambda x.E \\ K & \rightarrow & [\] \mid K E \mid v K \end{array}$$

$$\frac{}{(\lambda x.E) v \rightarrow E\{v/x\}}$$

$$\frac{E \rightarrow E'}{K[E] \rightarrow K[E']}$$

- Example

$$((\lambda x.x)(\lambda y.y))((\lambda z.z)(\lambda w.w)) \rightarrow (\lambda y.y)((\lambda z.z)(\lambda w.w))$$

$$K_1 = [\]((\lambda z.z)(\lambda w.w))$$

$$K_1[(\lambda x.x)(\lambda y.y)] = ((\lambda x.x)(\lambda y.y))((\lambda z.z)(\lambda w.w))$$

$$K_1[(\lambda x.x)(\lambda y.y)] \rightarrow K_1[\lambda y.y]$$

$$K_1[\lambda y.y] = (\lambda y.y)((\lambda z.z)(\lambda w.w))$$

$$(\lambda y.y)((\lambda z.z)(\lambda w.w)) \rightarrow (\lambda y.y)(\lambda w.w)$$

$$K_2 = (\lambda y.y)[\]$$

$$K_2[(\lambda z.z)(\lambda w.w)] = (\lambda y.y)((\lambda z.z)(\lambda w.w))$$

$$K_2[(\lambda z.z)(\lambda w.w)] \rightarrow K_2[\lambda w.w]$$

$$K_2[\lambda w.w] = (\lambda y.y)(\lambda w.w)$$

$$(\lambda y.y)(\lambda w.w) \rightarrow \lambda w.w$$

$$K_3 = [\]$$

$$K_3[(\lambda y.y)(\lambda w.w)] = (\lambda y.y)(\lambda w.w)$$

$$K_3[(\lambda y.y)(\lambda w.w)] \rightarrow K_3[\lambda w.w]$$

$$K_3[\lambda w.w] = \lambda w.w$$

Programming in the Lambda Calculus

- “Any computable functions are simulated by a Turing machine” (Church-Turing Thesis)
 - Equivalently, by the lambda calculus
- Concepts in high-level PL can be represented in the lambda calculus
 - E.g., Boolean, natural number, conditional expression, loop, recursion, etc
- Idea: value producer (introduction) / consumer (elimination)

Church Booleans

$$E \rightarrow \dots \mid \text{true} \mid \text{false} \mid \text{if } E \ E \ E \mid E \wedge E \mid E \vee E \mid \neg E$$

- Value producer: constants true and false
- Value consumer: test, negation, conjunction, disjunction, etc
- Intuition: boolean value = selector

$\text{true} = \lambda t. \lambda f. t$
 $\text{false} = \lambda t. \lambda f. f$
 $\text{if} = \lambda l. \lambda m. \lambda n. l \ m \ n$
 $\text{and} = \lambda b. \lambda c. b \ c \ \text{false}$
 $\text{or} = \lambda b. \lambda c. b \ \text{true} \ c$
 $\text{not} = \lambda b. b \ \text{false} \ \text{true}$

Example

- `if true v w`
- `and true true`
- `and true false`

Pairs

$$E \rightarrow \dots \mid \langle E, E \rangle \mid \text{fst } E \mid \text{snd } E$$

- Value producer: pairing of two values
- Value consumer: projection
- Intuition: pair = storage parameterized by selector

$$\text{pair} = \lambda f. \lambda s. \lambda b. b \ f \ s$$

$$\text{fst} = \lambda p. p \ \text{true}$$

$$\text{snd} = \lambda p. p \ \text{false}$$

Example

- $\text{fst}(\text{pair } v \ w)$

Church Numerals

$$E \rightarrow 0 \mid S \ E \mid E + E \mid E \times E \mid E^E$$

- Value producer: natural numbers
- Value consumer: successor, plus, times, etc
- Intuition: natural number = # function applications

$$\text{zero} = \lambda s. \lambda z. z$$

$$\text{one} = \lambda s. \lambda z. s \ z$$

$$\text{two} = \lambda s. \lambda z. s \ (s \ z)$$

$$\text{succ} = \lambda n. \lambda s. \lambda z. s \ (n \ s \ z)$$

$$\text{plus} = \lambda m. \lambda n. \lambda s. \lambda z. m \ s \ (n \ s \ z)$$

Loop

- Divergent combinator $\Omega = (\lambda x.x\ x)(\lambda x.x\ x)$
 - Cannot be evaluated to a normal form
- Fixpoint combinator

$$\text{fix} = \lambda f.(\lambda x.f\ (\lambda y.x\ x\ y))\ (\lambda x.f\ (\lambda y.x\ x\ y))$$

Recursion

- How can we define a recursive function using the lambda calculus?

```
let rec factorial n =  
  if n = 0 then 1  
  else n * factorial (n - 1)
```

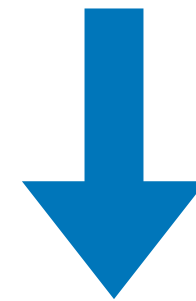
$$\text{factorial} = \lambda n. \begin{cases} 1 & \text{if } n = 0 \\ n \times \text{factorial}(n - 1) & \text{otherwise} \end{cases}$$

- Idea: fixpoint combinator

Definition as Fixpoint (1)

- Consider this inductive definition as an equation

$$\text{factorial} = \lambda n. \begin{cases} 1 & \text{if } n = 0 \\ n \times \text{factorial}(n - 1) & \text{otherwise} \end{cases}$$



$$\text{factorial} = \mathcal{F}(\text{factorial})$$

$$\text{where } \mathcal{F}(X) = \lambda n. \begin{cases} 1 & \text{if } n = 0 \\ n \times X(n - 1) & \text{otherwise} \end{cases}$$

Definition as Fixpoint (2)

- What is the definition of factorial? **A solution to this equation!**

$$\text{factorial} = \mathcal{F}(\text{factorial}) \quad \longleftrightarrow \quad X = \mathcal{F}(X) \text{ where } X = \text{factorial}$$

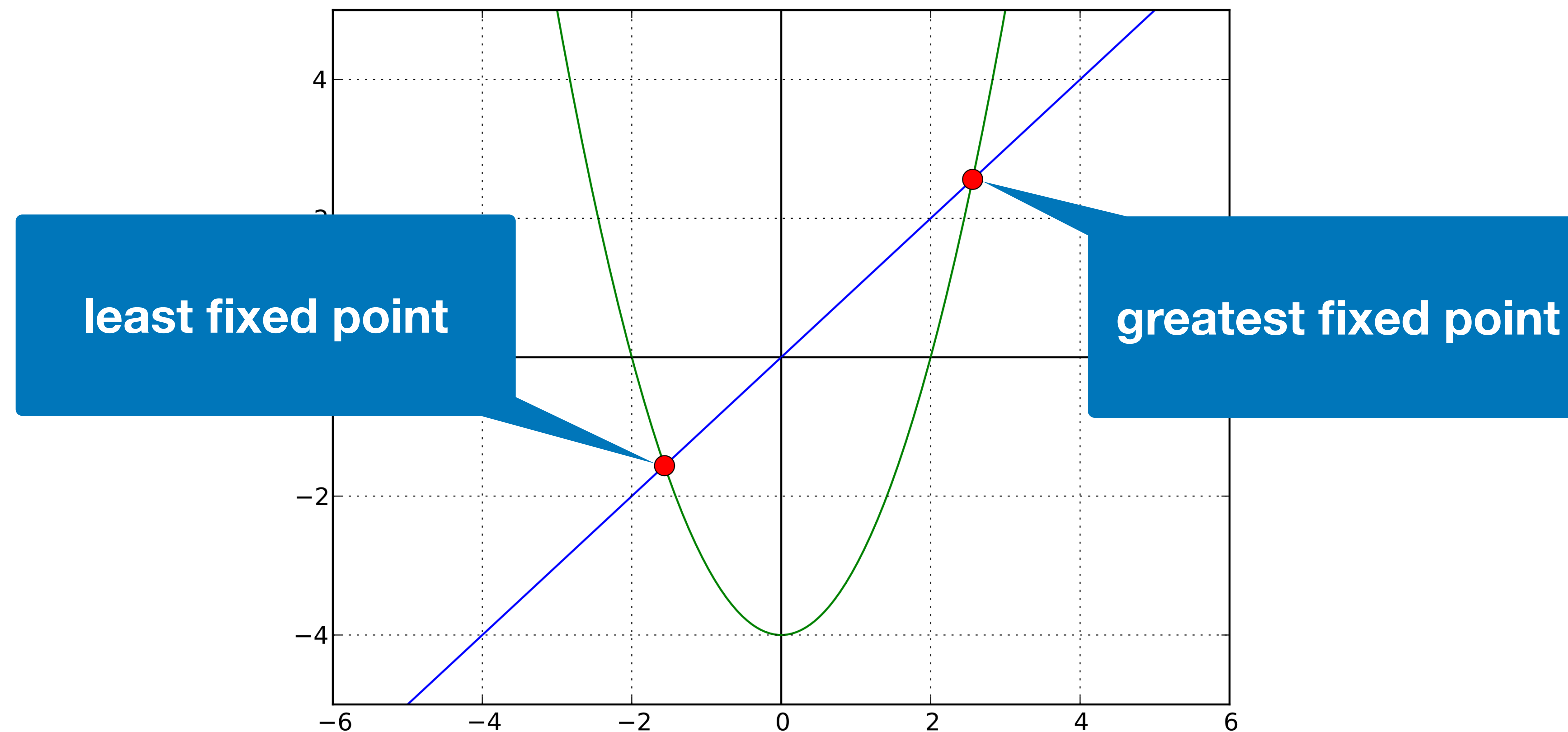
- Solution: a fixed point of $\mathcal{F}$

$$\text{factorial} = \text{fix } \mathcal{F}$$

$$\mathcal{F} = \lambda X. \lambda n. \text{if eq } n \ 0 \text{ then } 1 \text{ else times } n \ (X \ (\text{pred } n))$$

Exercise: Fixed Point

$$f(x) = x^2 - 4$$



*https://en.wikipedia.org/wiki/Least_fixed_point

Exercise: Fixed Point

$$\mathbb{N} = \{0\} \cup \{n + 1 \mid n \in \mathbb{N}\}$$

$\mathbb{N}$ is the least fixed point of F where $F(X) = \{0\} \cup \{n + 1 \mid n \in X\}$

$$\therefore \mathbb{N} = \text{fix}(\lambda X. \{0\} \cup \{n + 1 \mid n \in X\})$$

Exercise: Fixed Point

$$\mathbb{L} = \{\text{nil}\} \cup \{0 :: l \mid l \in \mathbb{L}\}$$

$\mathbb{L}$ is the least fixed point of F where $F(X) = \{\text{nil}\} \cup \{0 :: l \mid l \in X\}$

$$\therefore \mathbb{L} = \text{fix}(\lambda X. \{\text{nil}\} \cup \{0 :: l \mid l \in X\})$$

Exercise: Fixed Point

$$\text{reach}(N) = N \cup \text{reach}(\text{next}(N))$$

$$\text{reach} = \lambda N. N \cup \text{reach}(\text{next}(N))$$

reach is the least fixed point of F where $F(X) = \lambda N. N \cup X(\text{next}(N))$

$$\therefore \text{reach} = \text{fix}(\lambda X. (\lambda N. N \cup X(\text{next}(N))))$$

Exercise: Fixed Point

$$\text{fact}(N) = \text{if } N = 0 ? 1 : N * \text{fact}(N - 1)$$

$$\text{fact} = \lambda N. \text{if } N = 0 ? 1 : N * \text{fact}(N - 1)$$

fact is the least fixed point of F where $F(X) = \lambda N. \text{if } N = 0 ? 1 : N * X(N - 1)$

$$\therefore \text{fact} = \text{fix}(\lambda X. (\lambda N. \text{if } N = 0 ? 1 : N * X(N - 1)))$$

Summary

- Lambda calculus: foundation of programming languages
- Simple (variable, abstraction, application) but general enough (e.g., Church encoding)
 - Church-Turing thesis
- Various evaluation strategies
 - CBV: OCaml, C/C++, Java, etc
 - CBN: Haskell, etc